2026 IEEE International Parallel and Distributed Processing Symposium (IPDPS) | 979-8-3195-0602-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/IPDPS65963.2026.00085

KORAL: Knowledge Graph Guided LLM Reasoning for SSD Operational Analysis

Mayur Akewar†, Sandeep Madireddy‡, Dongsheng Luo†, Janki Bhimani†
†*Florida International University, Miami, FL, USA*
‡*Argonne National Laboratory, Lemont, IL, USA*
makew001@fiu.edu, smadireddy@anl.gov, dluo@fiu.edu, jbhimani@fiu.edu

Abstract*—Solid State Drives (SSDs) are critical to datacenters, consumer platforms, and mission-critical systems. Yet diagnosing their performance and reliability is difficult because data are fragmented and time-disjoint, and existing methods demand large datasets and expert input while offering only limited insights. Degradation arises not only from shifting workloads and evolving architectures but also from environmental factors such as temperature, humidity, and vibration. We present KORAL, a knowledge driven reasoning framework that integrates Large Language Models (LLMs) with a structured Knowledge Graph (KG) to generate insights into SSD operations. Unlike traditional approaches that require extensive expert input and large datasets, KORAL generates a Data KG from fragmented telemetry and integrates a Literature KG that already organizes knowledge from literature, reports, and traces. This turns unstructured sources into a queryable graph and telemetry into structured knowledge, and both the Graphs guide the LLM to deliver evidence-based, explainable analysis aligned with the domain vocabulary and constraints. Evaluation using real production traces shows that the KORAL delivers expert-level diagnosis and recommendations, supported by grounded explanations that improve reasoning transparency, guide operator decisions, reduce manual effort, and provide actionable insights to improve service quality. To our knowledge, this is the first end-to-end system that combines LLMs and KGs for full-spectrum SSD reasoning including Descriptive, Predictive, Prescriptive, and What-if analysis. We release the generated SSD-specific KG to advance reproducible research in knowledge-based storage system analysis. *GitHub Repository: Damrl-lab/KORAL

***Index Terms*—SSD, LLM, Ontology, Knowledge Graph**

I. Introduction and Motivation

Solid State Drives (SSDs) are the standard choice for fast and energy efficient storage in large scale datacenters [1], personal computers [2], and safety critical systems in aerospace and defense [3]. Their low latency, high throughput, and compact size raise the quality of service (QoS) for data intensive work such as web search, scientific computing, and real time analytics [4]. However, SSD behavior in the field varies with environmental conditions, workload characteristics, and design choices across vendors and product generations.

Environmental conditions strongly shape performance and endurance as temperature, humidity, and vibration vary across sites and seasons [5], [6]. Airflow and cooling create rack level thermal and moisture gradients, neighboring servers introduce platform vibration, and weather can push facilities beyond their preferred operating envelope. These stressors raise latency and error rates and can trigger incidents that propagate across the fleet [1], [7]–[12]. Controlled studies confirm that environmental stress elevates tail latency, reduces bandwidth, and increases the frequency of correctable and uncorrectable errors [3], [5], [6]. Workload characteristics modulate drive behavior since request size, queue depth, randomness, compressibility, and locality impose pressure on buffering, logical to physical mapping, and garbage collection [13]. Hardware and architecture add further freedom because flash technology and bits per cell shape program and erase error modes [8], [14], [15], controller design influences scheduling and protection [16]–[19], and the interface controls command handling and parallelism [20]–[22]. Caching strategy and power loss protection can shelter applications while introducing new tradeoffs [23], [24], and firmware policies for error correction, garbage collection, wear leveling, and over provisioning couple these elements into complex dynamics [16], [25], [26].

These factors shape SSD performance and reliability as production views report quarterly drift in annualized failure rates, which rise from 0.96% in 2023 Q1 to 1.05% in 2023 Q2 [27]–[29], and studies [5], [6], [13], [16], [17], [30]–[33] show that workload, environment, and design influence latency, throughput, error rate, and endurance. This motivates continuous structured analysis at device and fleet scale so that telemetry becomes timely insights and decisions that sustain target performance and reliability. Continuous monitoring reveals early drift, supports diagnosis of root causes [34]–[36], and guides interventions that stabilize latency, sustain throughput, and reduce error rates [37]–[39]. Operators can rebalance workloads across racks, adjust thermal policy during rising temperature, schedule firmware updates during low demand, and replace at risk drives before customer impact. These tasks remain challenging because the factors are interdependent, interactions are often nonlinear, and key signals evolve over time, while SMART indicators offer limited exploratory insight [40]. Many operational questions are distributional. An engineer may need to predict whether the 99th percentile latency next week will exceed a threshold during evening peak if rack temperatures rise by $2^{\circ}\mathrm{C}$ [37], [38].

Prior work addresses parts of this landscape through several complementary strands, as experimental campaigns [5], [6] deliberately vary temperature, humidity, and vibration in controlled settings in order to measure how tail latency and bandwidth respond to environmental stress and to quantify the margins at which performance begins to degrade, while

1530-2075/26/$31.00 ©2026 IEEE
DOI 10.1109/IPDPS65963.2026.00085

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

statistical workload studies [13], [41] analyze the distribution of request sizes, queue depths, read write ratios, randomness, and locality to uncover correlations with performance variability and to suggest operational knobs that practitioners can tune during incident response. A parallel body of investigations [42]–[47] mines SMART attributes and controller error logs to relate health indicators to impending failures and to shifts in performance, which enables threshold based alarms and risk scoring in production fleets, and operators regularly publish field reports [1], [11] that distill lessons from large telemetry repositories through careful manual inspection of health counters and workload traces in order to guide firmware upgrades, cooling policies, and replacement planning. Building on these descriptive efforts, many studies train machine learning models [43], [46], [48] on SMART features to predict failures or to assign risk scores, and in stable lab datasets these models often achieve attractive accuracy, which has motivated repeated attempts to bring such predictors into production environments. Extending this line of work, machine learning predictors [30], [48] combine long term and short term features [46], and rating methods compare mutation patterns of live drives with historical cohorts [47]; cross layer studies argue that models should join SMART, controller counters, and NAND wear indicators because SMART alone can miss important device phenomena [8], [41], [44], [49]; and generative or unsupervised detectors such as VAE, GAN, and autoencoders flag anomalies but often return scores that provide little guidance to operators [50]–[52].

Despite these advances, operational SSD telemetry analysis remains challenging. Manual workflows are time consuming, and machine learning pipelines often depend on ad-hoc feature selection. Training a joint model over heterogeneous telemetry is hard because labeled data are scarce: some signals are static, such as flash type and firmware algorithms, while others are dynamic, such as SMART, workloads, and environmental factors. As a result, model based approaches face heterogeneous and limited data, distribution shift across platforms and generations, and sparse labels for rare failures, which limits generalization. Many models cannot answer what-if questions or provide prescriptive recommendations that respect device constraints. Black box predictors also fail to show the facts behind their decisions. Experimental studies of environmental effects require specialized chambers and long campaigns, which makes them costly and slow to repeat, while proprietary traces and labels restrict open datasets, reproducibility, and progress. Finally, static fusion of SMART metrics, performance logs, and NAND errors struggles to separate workload noise from true degradation and offers little actionable guidance. There is a need for a framework that can analyze scattered telemetry holistically, deliver accurate predictive results, explain its reasoning with facts, produce clear descriptive summaries, offer prescriptive guidance to operators, and support what-if questions to aid decision-making.

One promising solution is to use Large Language Models (LLMs) [34], [35], [53]–[56], which can read diverse documents, connect evidence, and produce explanations aligned with engineering reasoning [54], [57], [58]. They can integrate research papers, white papers, postmortems, and facility logs, translating operator questions into structured steps [59]. However, applying LLMs to storage telemetry poses challenges: free-form generation can hallucinate [60], [61], correct reasoning requires rich context on device design, firmware, and operational constraints [62], and long time series must preserve ordering, units, and align events across devices and sites. Spatial and temporal relations must capture interactions among workload, environment, and design rather than averaging them away. Scale is also a hurdle, as fleets comprise thousands of devices generating numerous measurement streams. Addressing these issues requires explicit grounding, careful retrieval of authoritative evidence, and representations that make reasoning auditable and repeatable.

Motivated by the potential of LLMs to analyze SSD operational data in an effective, interpretable, and exploratory manner, and aiming to overcome the limitations of prior approaches while enabling accurate, hallucination-resistant analysis, we define the following **Research Questions (RQs)**: ***RQ1:*** *How can we collect and structurally represent established facts and results from existing studies so that they are reusable during operation.* ***RQ2:*** *How can we represent operational telemetry so that a model can summarize behaviour across space and time for individual drives and for entire fleets.* ***RQ3:*** *How can we provide the right context to reduce hallucination and to improve the faithfulness of answers.* ***RQ4:*** *How can we deliver grounded and complete exploratory analysis that spans descriptive, predictive, prescriptive, and what-if tasks.*

Our solution. We address these questions with a knowledge orchestrated reasoning framework that combines LLM with Knowledge Graph (KG) [63] grounded in an SSD taxonomy, in which literature and field evidence are distilled into entities, relations, and constraints aligned with a shared ontology [63], validated for consistency, and maintained as a living graph. Given a user query and recent operational data, the system issues a graph query to retrieve the pertinent subgraph and context, then generates evidence based explanations that support exploratory analysis for both device and fleet level operation, providing traceable reasoning and timely decision support even when data are sparse, heterogeneous, or partially missing. Our work makes the following contributions.

•*A knowledge orchestrated reasoning framework that unifies descriptive, predictive, prescriptive, and what-if analysis for SSD operation through the combination of a language model and a knowledge graph grounded in an SSD vocabulary.*

•*The first Literature KG focused on SSDs that is constructed from literature and operational sources and that is released to support reproducible research.*

•*Methods for concept extraction, taxonomy alignment, and fact validation that deliver transparent answers with citations and work with limited, heterogeneous data.*

•*An evaluation on production telemetry and diverse flash technologies that demonstrates expert level diagnosis and recommendations with markedly lower analysis time.*

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

To our knowledge **KORAL** is the first to deliver full spectrum SSD reasoning by combining an LLM with a data KG and it includes the first literature KG dedicated to SSDs.

II. Design Thinking

Operators need a method that turns fragmented SSD telemetry into answers that are faithful to evidence and easy to act on. We address this need by combining an intermediate representation with a domain KG so that a LLM can reason over grounded facts rather than raw streams.

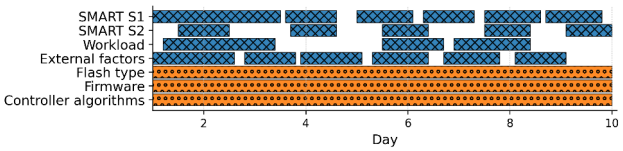


Fig. 1: Timeline of SSD signals across days by telemetry type. Cross-hatched bars denote dynamic, time-varying SMART telemetry (e.g., workload and external factors), while dotted bars denote static descriptors (e.g., flash type, firmware).

A. Elements

1) Data: SSD telemetry arrives in many forms and at different timelines. Homogeneous inputs include SMART attributes for a single drive over several days, which in our production dataset are sampled *daily* (the sampling interval is adjustable), workload features such as queue depth, request size, and burstiness, and environmental factors like $60°\mathrm{C}$ temperature and 50% humidity [6], [13], [31], [64]. Heterogeneous inputs combine sources, for example SMART with workload for a drive on a given day, SMART with environmental factors for a rack over two weeks, or SMART with workload and garbage collection traces for a busy application tier [16], [41], [44].

Signals appear at multiple granularities such as per drive, per drive per day, per drive across n days, fleet level per day, and fleet level across n days (Figure 1). Fields may be dynamic such as reallocated sector count, program fail rate, write amplification, and duty cycle, or static such as flash type, controller model, interface type, and firmware version. This diversity introduces mixed granularity, temporal misalignment, schema drift, and missing values that can obscure true relationships. A one day temperature spike cannot be compared to a fourteen day fleet average, and raw streams given to a LLM can yield ambiguous prompts where missing values are treated as zeros.

We address these issues by transforming telemetry into a stable intermediate representation of summaries that record mean, 95th percentile, trend slopes, change points, correlations, uncertainty intervals, and explicit coverage or missing data flags. For missing values, we apply simple imputation for common attributes with short gaps, attach explicit DataQuality nodes in the knowledge graph that mark missingness and imputation decisions, and downweight or drop windows with severe gaps so downstream reasoning does not treat missing as observed. We add semantic links among drives, sites, firmware, and environmental context to form a structured knowledge graph that preserves context, normalizes heterogeneous inputs, and enables scalable queries, and language models then reason over these compact grounded facts rather than raw logs.

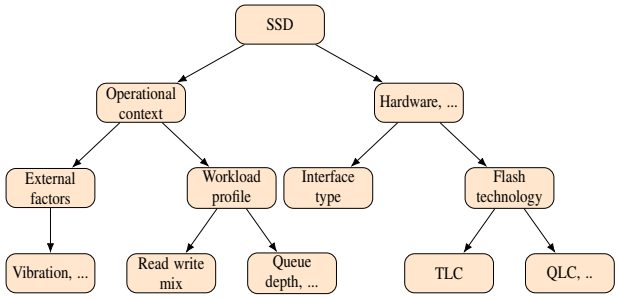


Fig. 2: A slice of the SSD taxonomy used as shared vocabulary. The taxonomy is seeded by experts; KORAL may propose new concepts from literature, which are added after validation.

2) Natural language and LLMs: LLMs excel at open vocabulary queries, pattern abstraction, hypothesis generation, and clear explanations [54], [56], [58]. They can integrate logs and metrics, learn patterns without task specific code, and support natural language queries for operators. Prior work demonstrates effective use for log analysis [36], [65], anomaly detection [36], root cause analysis [34], [35], scheduling and resource management [66], and storage tuning including hybrid SSD configuration [67]–[69]. Prompt strategies and in context learning such as LogPrompt [65] and AdaParser [70] improve parsing and diagnosis, hybrid designs enrich events and fine tune smaller language models such as BERT [71] as in LogLLM [36], and agent based studies explore root cause analysis with tool augmented LLMs [34], [35]. Knowledge graph reasoning and retrieval augmented generation connect signals, assets, and documents and recent surveys describe how these methods support global questions and explainable answers [56], while recent Graph RAG approaches explicitly build entity graphs and community summaries to answer global questions over large corpora [72], [73].

Direct application to raw telemetry remains challenging because outputs depend on prompt design and evidence order, advanced prompting strategies [74]–[76] require careful tuning, and meta prompt optimization increases cost, latency, and variability. Risks such as hallucination, context limits, privacy concerns, and data drift [77]–[79] further complicate production use. By reasoning over a structured knowledge graph, LLMs can produce faithful and hallucination resistant analysis from simple prompts, which enables descriptive, predictive, prescriptive, and what-if queries efficiently and reliably.

B. Context

1) SSD taxonomy: In this work we use a domain taxonomy of SSD together with a KG in order to support structured reasoning. The taxonomy supplies a shared vocabulary for devices, environments, workloads, and health signals, and the graph turns these concepts into entities and relations that can

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

be queried, validated, and summarized in natural language. The taxonomy organizes knowledge that operators already use in practice. At the top level it includes the operational context with external factors such as temperature, humidity, and vibration, the workload profile with read write mix and queue depth, the hardware stack with flash technology and interface type, the firmware layer with error correction, wear leveling, and garbage collection, and the metrics that describe performance and reliability. Figure 2 sketches a portion of the taxonomy that we use as a running vocabulary in the framework. The taxonomy is seeded by SSD domain experts, and KORAL only extends it by proposing out of vocabulary concepts automatically and adding them to the taxonomy after manual validation.

2) Ontology and knowledge graph: An ontology gives formal meaning to the vocabulary and constrains how concepts can be combined. We model an ontology as

$$\mathcal{O} = \big(C,\ R,\ I,\ A\big),$$

where C is a set of classes, R is a set of binary relations with declared domain and range, I is a set of individuals, and A is a set of axioms that include subclass assertions, typing assertions, and integrity constraints. In practice we express $\mathcal{O}$ with classes such as *EnvironmentalFactor*, *WorkloadAttribute*, *OperationType*, and *Metric*, and with relations such as *impactsOperation*, *impactsMetric*, *greaterImpactThan*, and *hasContext*. A KG is a set of typed entities connected by labeled edges that satisfy the ontology. Formally we write

$$\mathcal{G} = \big(V,\ E,\ \tau_V,\ \tau_E\big),$$

where V is a set of nodes, $E \subseteq V \times R \times V$ is a set of labeled edges, $\tau_V : V \to C$ assigns a class to each node, and $\tau_E : E \to R$ assigns a relation to each edge. When stored in Resource Description Framework form the graph is a set of triples $T \subseteq U \times U \times U$ over internationalized resource identifiers and literals, with semantics governed by RDFS [80] and OWL [81].

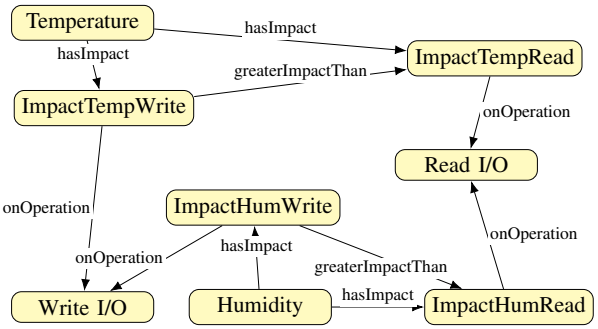


Fig. 3: Automatically generated Literature KG fragment from Stage I that encodes the claim that *"temperature and humidity impact write I/O more than read I/O"*. Nodes and relations are produced by the automated extraction and ontology alignment pipeline, with provenance linked to the supporting sentence.

Example graph. Consider the statement that *temperature and humidity changes impact the write I/O operations more than the read I/O operations*. Figure 3 shows one encoding that uses the taxonomy terms for temperature, humidity, and operations and the ontology relations introduced above. The nodes *ImpactTempWrite* and *ImpactTempRead* represent specific impact instances and the edge *greaterImpactThan* captures the comparative claim.

3) Storing and querying the graph: We store the graph in the Turtle format so that versions can be tracked and audited and so that the graph remains portable across tools. The snippet below mirrors Figure 3 and uses standard RDF and RDFS vocabulary [80].

```
@prefix ex: <http://example.org/ssd#> .
ex:Temperature ex:hasImpact ex:ImpactTempWrite .
ex:ImpactTempWrite
ex:greaterImpactThan ex:ImpactTempRead.
```

We query the graph with SPARQL [80], [81] in order to retrieve evidence for analysis. The example below asks for environmental factors that have a stronger impact on write operations than on read operations and returns both impact nodes.

```
PREFIX ex: <http://example.org/ssd#>
SELECT ?factor ?impactWrite ?impactRead
WHERE {?factor a ex:EnvironmentalFactor .
       ?factor ex:hasImpact ?impactWrite .
       ?factor ex:hasImpact ?impactRead .
       ?impactWrite ex:onOperation ex:WriteIO .
       ?impactRead  ex:onOperation ex:ReadIO .
       ?impactWrite ex:greaterImpactThan ?impactRead.}
```

The same pattern can be used with performance metrics. For example the relation *impactsMetric* can connect temperature to latency p ninety nine under a given workload profile, and a query can return the most relevant mitigation actions that are attached to the same subgraph.

a) Data quality validation.: Currently, missingness and imputation decisions are represented explicitly as DataQuality nodes and are used to downweight or drop low coverage windows during analysis. As future work, we plan to encode these validation rules as SHACL constraints, enabling declarative conformance checking of Data KG nodes before they are consumed by summarization and prompting.

III. KORAL Architecture

KORAL follows a two stage workflow (Figures 4 and 5). Stage I builds a literature grounded knowledge graph aligned to an SSD taxonomy and ontology, with validation and controlled updates. Stage II transforms heterogeneous telemetry into a rule based intermediate representation, materializes a Data KG, retrieves relevant literature evidence, and generates grounded SSD analysis that supports descriptive, predictive, prescriptive, and what-if queries.

A. Stage I: Building the Literature Knowledge Graph

Stage I constructs a Literature KG by extracting evidence backed claims from papers and reports and aligning them to a shared SSD vocabulary from the taxonomy and to typed

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

relations from the ontology (Figure 4). The output is an append only graph where each assertion retains provenance, including the evidence sentence and the source document identifier, so later analysis can cite supporting text and surface disagreements rather than overwriting them.

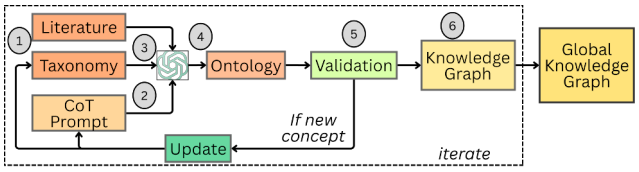


Fig. 4: KORAL Stage I Flow (RQ1). Document parsing, claim extraction, entity linking, and Literature KG construction are automated. When an out of vocabulary concept is detected, the system generates a concept proposal that is manually reviewed and validated before it is added to the vocabulary.

a) Inputs and preparation: We build a literature corpus from research and industry sources and convert each document into clean text with section anchors, so every extracted fact points to its supporting sentence. We pair this corpus with an SSD taxonomy that provides canonical anchors for external factors such as temperature and humidity, workload dimensions such as access pattern and queue depth, and metrics such as IOPS and the 99th percentile latency. Each document is assigned a stable identifier to ensure traceability during graph merging into a global KG.

b) Prompt guided extraction and ontology alignment: A language model reads the full document under taxonomy guidance and returns a strict JSON containing entities, triples, mappings, axioms, and optional concept proposals. The prompt enforces taxonomy first mapping by linking each mention to the closest taxonomy path and preferring specific children such as 99th percentile latency over generic parents. It distinguishes classes from instances, requires explicit context that connects an SSD instance to environment and optionally to a workload profile, and uses directional predicates when a paper reports improvement or degradation. Each triple carries an evidence sentence and a confidence score to preserve provenance and to support auditing.

We align extracted concepts to ontology classes (for example EnvironmentalFactor, WorkloadAttribute, OperationType, and Metric) and relations (for example impactsOperation, impactsMetric, causes, mitigatedBy, and greaterImpactThan), and we normalize synonyms such as tail latency and p99 latency to the same metric class. This alignment ensures that queries over the graph remain consistent across sources and devices.

c) Validation, evidence alignment, and versioned materialization: Stage I enforces a validation gate before any extracted claim is committed to the Literature KG. We validate each JSON output for schema completeness, apply ontology type checks to ensure every triple has compatible subject and object types, and align each triple to an explicit evidence span in the source text. Triples that fail schema or type constraints, lack evidence alignment, or fall below a configurable confidence threshold are rejected. Units and percentiles are standardized where applicable, and contradictory claims are retained with distinct provenance but down weighted during retrieval so higher confidence, better supported evidence is preferred. Validated triples are written in Turtle form and versioned in a repository, and we maintain both per collection graphs and a merged global graph so knowledge accumulates safely over time.

d) Worked example: Consider the sentence that a high inlet temperature raises the 99th percentile latency under a write heavy workload. The extractor maps this to the entities Temperature, 99th percentile latency, and Write heavy, and encodes a relation that connects the environmental factor to the latency metric under the specified workload condition. Under the prompt constraints, the claim becomes a typed assertion with a directional predicate (for example `degrades`), includes the evidence sentence verbatim, and binds the entities to taxonomy paths so that later graph queries can retrieve and reuse the claim consistently.

e) Vocabulary growth under controlled review: When a document introduces a concept not covered by the current taxonomy, the prompt asks the model to propose a new concept with a short definition and a suggested parent. The pipeline records these proposals for review, so the vocabulary can evolve without breaking existing mappings and without silently changing meaning across versions.

f) Outcome: Stage I yields a Literature KG that distills entities, relations, constraints, and provenance from the corpus, aligned to the SSD taxonomy and validated for consistency. This graph serves as the reusable evidence base for Stage II during operational analysis.

B. Stage II: Operational SSD Analysis

Stage II begins from the available telemetry and selects an intermediate representation using a rule base (Figure 5). The rule base is an expert curated repository of summarization and mapping rules, and it is not automatically learned from data. The rule base defines how to summarize dynamic signals such as SMART, workload, and environmental factors across drive and fleet granularity within an n day window, while passing static descriptors such as flash technology, controller model, and firmware as stable facts. The goal is to preserve temporal structure and data quality information while producing compact, typed frames that can be turned into a Data KG.

a) Intermediate representation and frame emission: The pipeline computes robust statistics and temporal features such as trends, rate of change, change points, seasonality, correlation, exposure above thresholds, and explicit coverage and missingness indicators. It emits typed frames with stable identifiers and provenance, including AttributeFrame for SMART summaries, WorkloadFrame for workload profiles, EnvFrame for environmental summaries, and DataQuality frames that record missingness, sensor checks, and imputation. It also emits Episode or Observation frames when multi day regime shifts or cross source associations are detected.

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

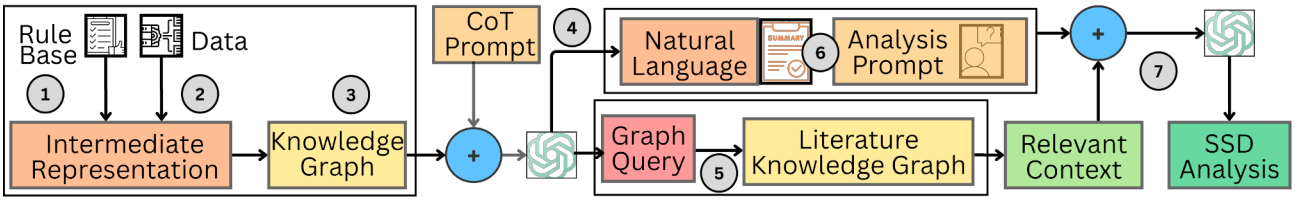


Fig. 5: KORAL Stage II Flow (RQ2, RQ3 & RQ4). The only expert authored component is the curated rule repository that defines how telemetry is summarized into frames and mapped to graph relations. Given this rule base, intermediate representation construction, Data KG creation, node linking, summarization, retrieval, and generation are automated.

b) Worked example: For a drive window, the system can create an AttributeFrame for percentage used that includes median, 95th percentile, rate of change per day, Mann Kendall trend statistic and p value. It can also create a WorkloadFrame with read share, average queue depth, and burstiness, and a DataQuality frame that reports partial coverage such as five observations out of seven with one imputed value. If a change point detector flags a regime shift starting on day four, the pipeline records this as an Episode linked to the same window identifier so downstream reasoning can separate transient spikes from persistent drift.

c) Materializing the data knowledge graph: We build the Data KG by converting frames into typed nodes with window tags, computation methods, confidence, and provenance, and by instantiating relation templates from the rule base. For example, SMART summaries become AttributeFrame nodes connected to the corresponding drive and window, workload and environment frames connect through hasContext style edges, and Episodes link to the signals that triggered them. This graph provides a compact operational context that replaces raw time series during downstream analysis while keeping traceability intact.

d) Grounded summarization, retrieval, and answer generation: Given the Data KG, KORAL generates a natural language summary that preserves units, time windows, uncertainty, and data quality statements. In parallel, it produces a graph query over the shared vocabulary to retrieve a relevant subgraph from the Literature KG. The final prompt combines the user question, the Data KG grounded summary, and the retrieved literature context, then generates descriptive, predictive, prescriptive, or what-if outputs depending on the query intent and available frames.

e) Audit trail: To support reproducibility and debugging, KORAL stores the emitted frames, the Data KG identifiers, the literature query, retrieved evidence, and the final response as artifacts tied to the same window tag, enabling later inspection of what was observed, what was retrieved, and what the model claimed. When the operator accepts a recommendation the system can attach a mitigation action node to the data graph for that window and can schedule a follow up query that checks whether the expected improvement in latency and error rates occurred during the next week, which closes the loop between observation, literature grounded reasoning, and operational action.

f) Automation versus expert authored components: KORAL is fully automatic at runtime. The only expert authored artifact is the curated rule repository, which specifies attribute groupings, ideal directions, windowing and summarization functions, and the mapping from frames to typed graph relations. All other modules, including intermediate representation construction, Data KG materialization, node linking, summarization, literature retrieval, and response generation, execute automatically given telemetry and the rule repository. In Stage I, the pipeline can automatically flag out of vocabulary concepts, but they are only added to the vocabulary after manual validation.

C. Running Example: From Telemetry to Prompt

Figure 5 outlines Stage II at a high level. We now trace a single drive through the full pipeline using a 30 day SMART window (daily samples) and a representative subset of attributes. Each SMART attribute is also tagged with an *ideal* direction (Low, High, or Monitor) to preserve polarity for later scoring and explanations.

a) Raw telemetry (30 day SMART window): For a drive d, we ingest a daily SMART record x_t containing counters such as reallocated sectors (`r_5`), wear and erase indicators (`r_173`, `r_177`), program and erase failures (`r_181`, `r_182`), uncorrectable errors (`r_187`), pending sectors (`r_197`), CRC errors (`r_199`), and traffic counters such as total LBAs written and read (`r_241`, `r_242`). The rule base also records coverage (how many days observed), missingness, and simple validity checks (for example counter monotonicity).

b) Intermediate representation (SMART attribute frames): The rule base converts each attribute time series $\{x_t\}_{t=1}^{30}$ into an *AttributeFrame* that preserves temporal structure and uncertainty in a compact form. Each frame stores robust summaries and trend descriptors, for example median, tail percentiles, slope, volatility, change points, and exposure beyond operator meaningful thresholds, together with metadata such as the attribute id, units, and the ideal direction. A schematic example frame is shown below.

```
AttributeFrame(
  drive_id = "Disk/26871",
  window   = "2019-11-09..2019-12-08",
  attr_id  = "r_187", % Reported_Uncorrectable_Errors
  ideal    = "Low",
  summary  = {median, p95, last, max},
```


Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

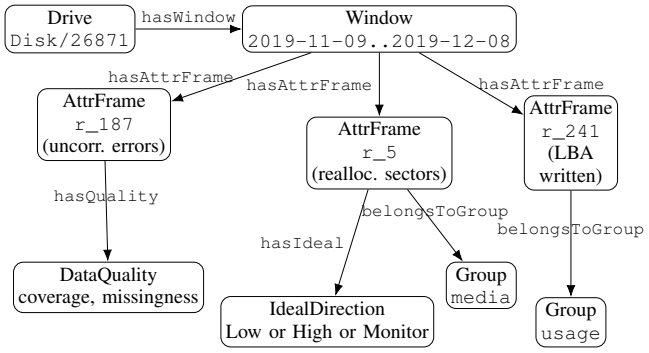


Fig. 6: Schematic Data KG for one drive and a 30 day SMART window. Applying the curated rule repository to raw telemetry automatically creates AttributeFrame and DataQuality nodes and their links, enabling traceable claims with units, window scope, and coverage.

```
temporal = {slope, change_point?, spike_days?},
quality  = {coverage=30/30, missing_days=0})
```

c) Data KG materialization (how the graph looks): Frames are then materialized into a Data KG where each node is typed and each edge is auditable. Figure 6 shows a readable subgraph for one drive and one window. The drive node links to the window node; the window links to AttributeFrames for selected SMART attributes; each attribute frame links to a data quality node, and to a semantic group node (media, interface, usage) so later retrieval and prompting can reference families rather than long attribute lists. This structure is what enables traceability, because every generated claim can be mapped back to a specific frame and window.

d) LLM prompt assembly (what the model actually sees): Finally, KORAL converts the Data KG subgraph into a compact, structured context and optionally augments it with retrieved evidence from the Literature KG using the same vocabulary (attribute names, workload concepts, and metrics). The prompt includes (i) the user query, (ii) the Data KG summary with units and coverage statements, and (iii) retrieved literature snippets with citations. A simplified prompt skeleton is shown below.

```
Task: Provide a grounded SSD analysis for Disk/26871
over window 2019-11-09..2019-12-08.
Data KG context (auditable):
- r_187 Reported_Uncorrectable_Errors: trend=?,
change_point=?, coverage=30/30
- r_5   Reallocated_Sector_Count: trend=?,
spikes=?,  coverage=30/30
- r_241 Total_LBA_Written:
burstiness=?, coverage=30/30
- Data quality: missing_days=0, notes=...
Retrieved literature (if available):
- Evidence linking thermal or workload
stressors to tail latency or error behavior,
with citations.
User query:
"Explain the risk signals and suggest mitigations.
Provide a what if comparison if workload is reduced."
```

This running example illustrates how Stage II transforms a 30 day stream of SMART counters into a structured intermediate representation, materializes it as a typed Data KG with window level provenance, and then converts the relevant subgraph into a compact prompt context that can be augmented with retrieved literature evidence for grounded analysis.

IV. Evaluation

This section defines the datasets, evaluation protocol, metrics, and results. We evaluate KORAL on individual SSD telemetry data, which includes SMART attributes [4], [44], workload counters [4], [82], and environmental measurements [5], [6]. We also assess heterogeneous combinations of these signals by augmenting dynamic telemetry with static device metadata. We evaluate performance and reliability at device and fleet levels in an exploratory analysis covering descriptive, predictive, prescriptive, and what-if scenarios.

A. Datasets and Setup

SMART telemetry, workload context, and failures. We use two complementary SSD corpora. The Google field study spans millions of drive days across six years, about ten device models, and multiple flash types [44], and we use it to guide attribute selection, quality checks, and long horizon drift handling [4]. The Alibaba dataset provides daily snapshots from January 2018 to December 2019 for more than five hundred thousand drives across six SSD models (MA1, MA2, MB1, MB2, MC1, MC2), with confirmed failure tickets, rack or site identifiers, and application tags [4]. Each snapshot reports up to 105 SMART attributes per drive, including vendor normalized scores `n_i` and raw counters `r_i`, together with device id, timestamp, and model code, enabling analysis of absolute error counts, wear indicators, and cross model differences [4], [40]. We keep the 15 raw attributes and treat each drive's history as a multivariate daily time series. We focus on the **`MB1`** and **`MB2`** models in our experiments because they have very few missing values for each of the 15 attributes and are also used by prior work for evaluation making it straightforward to compare.

Environmental signals and modality alignment. We incorporate environmental time series from two controlled studies [5], [6] that report temperature in °C, relative humidity in %, and vibration frequency and amplitude, together with measured throughput and tail latency percentiles. We preserve units and provenance fields (study, run, and configuration) and retain metadata such as device model, firmware revision, sampling interval, and exposure duration for reproducibility. We evaluate five modality settings: SMART plus Workload, SMART plus Environmental, Workload plus Environmental, SMART plus Workload plus Environmental, and the same joint set augmented with static descriptors (for example flash type and firmware). For SMART plus Workload we use Alibaba traces [4] whose workload taxonomy includes eight categories: Web Service Management (WSM), Resource Management (RM), Web Proxy Services (WPS), SQL Services (SS), Database (DB), Web Services (WS), Data Analytics Engine (DAE), and Network Attached Storage (NAS). Because

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

few traces contain all signals for the same device and window, device level joint analysis uses paired or tripled windows by aligning Alibaba SMART and workload windows with the environmental conditions in [5], [6] using window keys and strict timestamp tolerance, then filtering by coverage and monotonicity checks. For Workload plus Environmental tests, we generate workload profiles using FIO [82] following published request size and concurrency settings and combine them with the environmental conditions from [5], [6]. For fleet level analysis we avoid synthetic joins and rely on natively aligned Alibaba SMART and workload telemetry [4], and we do not incorporate environment at fleet scale because these feeds cannot be synchronized to the production traces.

Protocol, prompts, and reference text. We follow the Stage II pipeline (Sec. III-B) to build the intermediate representation and Data KG, retrieve context from the Literature KG, and generate outputs using GPT 4o [83]. Reliability experiments use 30 day SMART windows with ground truth labels for failure within a fixed horizon and time to failure defined as days from the window end to the failure event. At the drive level, for each window we issue four prompts covering predictive, descriptive, prescriptive, and what-if tasks, then aggregate metrics across windows as defined in Sec. IV-B. At the fleet level, we ingest each trace as a cohort, treat predicted failing drives as individual predictions for classification against ground truth, and generate one predictive, one descriptive, and one what-if output per query, then aggregate metrics across query types. For natural language evaluation, we create reference summaries and rationales using GPT 4o [83] as a drafting assistant by supplying evidence passages from [4]–[6], [44] together with representative intermediate representations; we manually verify and edit all references for fidelity and remove unsupported statements.

B. Evaluation Metrics

We evaluate KORAL along two axes: numeric prediction for reliability and performance, and LLM generated text for descriptive summaries, prescriptive recommendations, and what-if analysis. Results in Table I and Table II report classification and regression metrics for prediction, and BLEU and ROUGE plus grounding metrics for text.

a) Reliability prediction (classification): For each drive d and analysis window ending at time t, the binary ground truth failure label $Y_{d,t}^{\text{fail}} \in \{0,1\}$ indicates whether the drive fails within the evaluation horizon after the window, and the model outputs $\hat{Y}_{d,t}^{\text{fail}} \in \{0,1\}$. We report *Precision* $\mathbf{P} = \frac{\text{TP}}{\text{TP+FP}}$, *Recall* $\mathbf{R} = \frac{\text{TP}}{\text{TP+FN}}$, and *Accuracy* $\mathbf{A} = \frac{\text{TP+TN}}{\text{TP+FP+FN+TN}}$ [84], [85]. In our SSD setting, True Positive (TP) is a window correctly flagged as at risk for a drive that fails within the horizon, False Positive (FP) is an unnecessary alarm on a healthy drive window, False Negative (FN) is a missed warning for a drive that fails within the horizon, and True Negative (TN) is a correctly identified healthy window.

b) Reliability and performance estimation (regression): We also evaluate continuous targets with mean squared error. For time to failure, $T_{d,t}^{\text{ttf}} > 0$ is the number of days from the end of window t until the recorded failure event, and we report $\text{MSE}_{\text{TTF}} = \frac{1}{N}\sum_{i=1}^{N}\big(T_i^{\text{ttf}} - \hat{T}_i^{\text{ttf}}\big)^2$. For example, if a drive fails 18 days after the window ends but the model predicts 25 days, the squared error for that window is $(18-25)^2 = 49$ days^2, and MSE averages this value across all evaluated windows. For performance, we predict tail latency $L_{d,t}^{(p)}$ (the p percentile latency in milliseconds) and report $\text{MSE}_{\text{TL}} = \frac{1}{N}\sum_{i=1}^{N}\big(L_i^{(p)} - \hat{L}_i^{(p)}\big)^2$. For example, if the observed $p99$ latency under a given temperature and workload condition is 3.2 ms and the model predicts 2.6 ms, the squared error is $(3.2-2.6)^2 = 0.36$ ms^2, and MSE averages this error across all windows and runs.

c) Text quality and grounding: For descriptive, prescriptive, and what-if outputs, we measure surface overlap against curated references using BLEU 4 [86] and ROUGE L [87]. BLEU 4 is computed from modified n gram precisions up to $n = 4$ with a brevity penalty, $\mathbf{B4} = \text{BP} \cdot \exp\big(\sum_{n=1}^{4} w_n \log p_n\big)$. ROUGE L is based on the longest common subsequence between a generated text and the reference and is reported as an F_1 score [87]. Because overlap does not guarantee factual correctness, we also report task aligned grounding metrics. For descriptive summaries and prescriptive recommendations we use Faithfulness Precision, $\mathbf{FiP} = \frac{\#\text{supported atomic claims}}{\#\text{atomic claims}}$, where a claim such as "Percentage_Used rose by 12% over 30 days" counts only if it is verified from the intermediate representation or Data KG nodes for the same drive and window, including units and coverage. For what-if analysis we use Counterfactual Validity, $\mathbf{CFV} = \frac{\#\text{counterfactual statements with evidence consistent direction}}{\#\text{counterfactual statements}}$, for example "if inlet temperature decreases by 5°C, tail latency decreases" counts only when cohort evidence or retrieved literature supports that directional association under comparable conditions.

C. Results

We evaluate the operational implications of KORAL's design choices, including the intermediate representation, the Data Knowledge Graph, the Literature Knowledge Graph, and modality fusion. Device level results are reported in Table I and fleet level results are reported in Table II, while Figure 7 compares reliability prediction against learned baselines on Alibaba SMART windows and Figure 8 quantifies the impact of major components via ablation.

1) Reliability prediction across scale and datasets: At the device level, KORAL improves failure occurrence precision and accuracy and reduces time to failure error because the intermediate representation retains daily trends, change points, and uncertainty that would otherwise be flattened. Recall increases when workload or environment are added to SMART since the model can distinguish short spikes from sustained drift and relate known stressors to elevated risk. Fleet level prediction is inherently harder because one decision aggregates many devices and operating regimes, so precision and recall are lower than at the device level, yet adding workload to SMART at fleet scale still improves reliability metrics and

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

TABLE I: SSD Operational Data Analysis (**Drive Level**). Abbreviations: **FT** = Flash Type, **Al** = Algorithms. **P** = Precision (%), **R** = Recall (%), **A** = Accuracy (%), **TTF-MSE** = Time-to-Failure-Mean Square Error (days), **TL-MSE** = Tail Latency-Mean Squared Error (%), **B4** = BLEU-4, **RL** = ROUGE-L, **FiP** = Faithfulness Precision, **CFV** = Counterfactual Validity, **__ = NA**.

Dataset Type	**Dataset Used**	**Predictive**					**Descriptive**			**Prescriptive**			**What-if**		
		P ↑	**R** ↑	**A** ↑	**TL-MSE**↓	**TTF-MSE**↓	**B4** ↑	**RL** ↑	**FiP** ↑	**B4** ↑	**RL** ↑	**FiP** ↑	**B4** ↑	**RL** ↑	**CFV** ↑
SMART	Alibaba	**96**	41	92	—	20	65	73	79	62	71	74	59	70	67
SMART	Google	92	38	90	—	23	63	72	77	60	70	72	57	69	65
Environmental (Env.)	Exp. Studies	—	—	—	**2.9**	—	64	73	78	62	71	74	66	73	79
SMART+Workload	Alibaba	94	49	93	—	**15**	68	75	82	65	73	78	63	72	75
SMART+Env.	Alibaba + Exp. Studies	95	52	**94**	3.2	20	**69**	**76**	**83**	66	**74**	**80**	66	**74**	79
Workload+Env.	FIO + Exp. Studies	—	—	—	4.2	—	65	74	79	63	72	77	62	71	76
SMART+Workload+Env.	Alibaba + Exp. Studies	**96**	**58**	**94**	3.0	**15**	61	68	76	**69**	66	72	60	67	73
SMART+FT	Alibaba	95	45	93	—	22	66	74	81	63	72	77	62	72	76
SMART+Al	Alibaba	**96**	47	93	—	25	66	75	82	64	73	78	64	73	78
Env.+FT	Exp. Studies	—	—	—	3.6	—	66	75	82	64	73	79	66	**74**	**81**
Env.+Al	Exp. Studies	—	—	—	3.5	—	67	**76**	**83**	65	74	**80**	**67**	75	**82**

TABLE II: SSD Operational Data Analysis (**Fleet Level**). [Abbreviations as above]

Dataset Type	**Dataset Used**	**Predictive**				**Descriptive**			**Prescriptive**			**What-if**		
		P ↑	**R** ↑	**A** ↑	**TTF-MSE** ↓	**B4** ↑	**RL** ↑	**FiP** ↑	**B4** ↑	**RL** ↑	**FiP** ↑	**B4** ↑	**RL** ↑	**CFV** ↑
SMART	Alibaba	82	32	89	30	62	69	73	62	65	64	55	70	67
SMART	Google	85	37	91	35	63	65	70	60	69	63	51	69	65
SMART+Workload	Alibaba	**87**	**41**	**93**	**24**	**67**	**73**	**77**	**64**	**71**	**74**	**61**	**70**	**71**

lowers time to failure error, indicating that the representation and aggregation preserve useful signal at scale.

Generalization is evaluated across Alibaba and Google cohorts. SMART only prediction transfers without retuning, which suggests that the intermediate representation plus Data Knowledge Graph (KG) yield stable features despite vendor naming differences and noise. Across cohorts, adding workload or environmental context consistently raises recall with minimal precision change, and it also improves descriptive, prescriptive, and what-if metrics, showing that richer context helps while robustness is maintained across sources.

2) Impact of modality fusion: Adding workload to SMART increases recall and improves the faithfulness of natural language outputs because workload burstiness and queue depth explain shifts in error counts and latency. For example, under SQL Services workload, daytime concurrency spikes and nightly index rebuild behavior create queue depth surges and higher 95th percentile latency, and with workload context the model interprets brief error bursts as pressure episodes rather than immediate failure.

Adding environmental context improves recall and tail latency estimation because the intermediate representation encodes temperature and humidity with units and links them to attribute frames. During a chiller switchover, inlet temperature rises from 24°C to 31°C for about four hours with relative humidity near 60%, and the graph links this excursion to higher latency and correctable error spikes.

Using SMART, workload, and environment together yields the best predictive performance and the strongest grounding, achieving precision of 96% and recall of 58% at the device level while also reducing time to failure error, since the generator can cite operational load and ambient conditions for each claim instead of relying on implicit assumptions.

3) Grounded natural language analysis and the roles of the two knowledge graphs: The Data KG is the primary source of factual content for both prediction and descriptive summaries because each claim can be traced to an intermediate representation frame with units, window scope, and coverage, which improves BLEU and ROUGE as well as Faithfulness Precision. This structure rewards grounded statements that connect recent trends and co occurring signals, for example increases in uncorrectable errors over the last days paired with concurrent shifts in controller level behavior, because each atomic claim can be checked against the corresponding attribute frame.

Retrieval from the Literature KG contributes most strongly to prescriptive and what-if quality because it supplies mechanism level evidence and constraints that justify actions and counterfactual directions. On Alibaba SMART at the device level, prescriptive Faithfulness Precision improves from 74 with SMART alone to 80 with SMART plus environment, and Counterfactual Validity increases from 67 to 79 in Table I. At the fleet level, moving from SMART to SMART plus workload raises prescriptive Faithfulness Precision from 64 to 74 and Counterfactual Validity from 67 to 71 in Table II. These gains occur when graph queries retrieve studies on thermal sensitivity, humidity effects, and vibration coupling, enabling KORAL to reuse validated mechanisms and thresholds while keeping telemetry grounded in the Data Knowledge Graph.

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

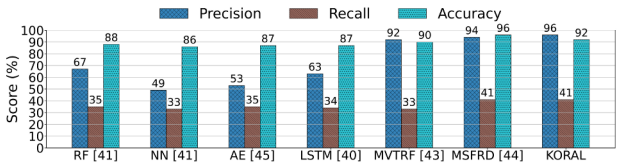


Fig. 7: Comparison with baselines on Alibaba SMART data.

4) Comparison, ablation, and a end to end example: As shown in Figure 7, KORAL surpasses learned baselines on Alibaba SMART data with 30 day windows by preserving temporal detail and reasoning over multiple modalities through an intermediate representation and two knowledge graphs. Random forest on raw SMART ignores temporal evolution and struggles under drift with weaker recall [44], the feed forward neural network on raw SMART is sensitive to class imbalance and drift and needs frequent retuning while lacking explicit temporal structure and operator facing health cues [44], and the LSTM needs long well labeled histories, yields opaque scores for rare failures, and still operates as a single view learner that trails MVTRF and MSFRD [43], [46], [47]. Even MVTRF and MSFRD require feature curation and periodic retuning as distributions shift [46], [47]. KORAL preserves daily trends and change points in a rule based intermediate representation, records units and coverage in the Data KG, fuses SMART with workload and environment without retraining, and grounds recommendations and counterfactuals using retrieved literature. This modality aware pipeline improves precision and accuracy, boosts recall when context is added, and reduces error for time to failure and tail latency. By combining paired Data and Literature knowledge graphs with explicit provenance and citations, KORAL supports predictive, descriptive, prescriptive, and what-if analysis across modalities without retraining.

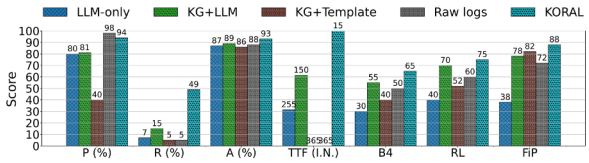


Fig. 8: KORAL Ablation: end-to-end impact of IR and knowledge grounding on failure prediction, TTF estimation, and analysis quality (Abbreviations as in Table I).

We ablate KORAL on Alibaba SMART plus Workload with a thirty day window as shown in Figure 8. Time to failure (TTF) is normalized and inverted to align with the axis. Text metrics are reported as a combined analysis score by sampling prompts from descriptive, prescriptive, and what-if tasks and averaging over windows. Across ablations, partial pipelines tend to be overly conservative: precision can remain high, but recall collapses, TTF error grows, and the generated analyses become less consistent. In particular, KG plus LLM mainly improves the readability and coverage of generated summaries, while KG plus Template increases faithfulness oriented scores at the cost of weaker failure sensitivity, and raw logs behaves like hard thresholding that misses many failures.

In contrast, KORAL is consistently strongest across both predictive and generative axes. With all components enabled, it maintains high precision while substantially improving recall, achieves the best overall accuracy, and yields the lowest TTF error, indicating earlier and more reliable failure anticipation. It also produces the most informative and faithful analyses, leading on BLEU 4, ROUGE L, and Faithfulness Precision, which reflects better instruction following and background preservation in the generated reports. These gains come from the synergy of the intermediate representation with the Data KG and Literature KG: the IR captures temporal structure and change points, the Data KG anchors numeric claims and cohort context, and the Literature KG adds mechanism level grounding that strengthens prescriptive and counterfactual reasoning.

Example. Consider drive D42 at site Alpha in a window where the summary records a temperature rise from 40°C to 58°C, an increase in write share from about 60% to 80%, a higher program fail rate, step increases in write amplification aligned with workload changes, and missing airflow on two days. KORAL encodes these as typed frames and graph nodes linked to the device and window, then retrieves literature about thermal stress under write heavy workloads. The resulting output can explain the observed latency and error behavior using data grounded evidence, propose operational actions such as reducing write burst intensity or improving airflow when retrieved evidence supports it, and answer what-if queries such as the expected change if queue depth is capped or ambient temperature is reduced, while keeping each claim traceable to either the Data KG or retrieved literature context.

V. Related Work

Prior work on SSD operational analysis and system logs are diverse, covering environmental impacts, SMART-based reliability and failure prediction, and broader observability and performance diagnosis.

Environmental impact and operating conditions. A line of work shows that external conditions materially affect SSD behavior and reliability. Experimental studies examine how temperature, humidity, and vibration exposures influence latency, performance variability, and degradation trends [5], [6]. Complementary efforts focus on thermal effects inside the device, including predicting throttling behavior [88] and improving NAND reliability by explicitly exploiting temperature awareness and self-recovery properties [89]. Beyond immediate reliability, sustainability-oriented studies highlight the broader environmental footprint of storage devices, including embodied carbon considerations [90]. Most recently, LLMs have been explored as a modeling layer to connect environmental factors with SSD outcomes and summarize multi-factor impacts from heterogeneous evidence [58]. These works motivate KORAL's explicit treatment of environment as first-class operational context, but they typically do not unify

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

environmental signals with workload, SMART telemetry, and literature knowledge in one structured reasoning pipeline.

SMART telemetry and SSD failure prediction. A substantial literature uses SMART attributes and fleet telemetry to characterize failure modes and build predictive models. Large-scale field studies report failure symptoms, root causes, and operational failure patterns at scale [8], [9], [20], [41], [44], [49], while postmortem analyses of SSD-related incidents highlight practical lessons from real deployments [11]. Surveys summarize the broader design space of lifecycle prediction and modeling choices [30]. On the modeling side, feature selection and supervised learning remain common for failure prediction in large deployments [45], and more recent approaches improve adaptability and interpretability of predictors [43], [46]–[48]. While these methods can achieve strong predictive accuracy, they often output scores/labels with limited operator-facing explanation, and they generally do not connect predictions to structured evidence from documentation and prior studies in a way that supports *descriptive, prescriptive, and what-if* operational queries.

LLMs for operational reasoning and system logs. Recent systems increasingly treat LLMs as an "ops copilot" that can read noisy operational artifacts, including logs, traces, and configuration metadata, and produce summaries, anomaly hypotheses, and remediation suggestions [36], [53], [54]. A parallel thread explores tool-augmented (often agentic) incident workflows, where the LLM is coupled with log search, dashboards, and code/context inspection to support root-cause analysis and incident triage, including multi-step and multi-agent variants for verification and action planning [34], [35]. Beyond diagnosis, LLMs have also been used to navigate large tuning/control spaces: e.g., guiding HPC I/O optimization and configuration exploration, assisting database/LSM tuning, and pairing natural-language goal specification with policy learning for scheduling and control in heterogeneous environments [55], [61], [66]–[68]. Within storage management, initial work applies LLMs to high-level tiering and parameter selection for hybrid SSD deployments [69], while recent efforts study LLM+RAG pipelines to connect environmental factors with SSD performance and reliability outcomes from heterogeneous evidence [58]. However, log-centric LLM pipelines can still over-generalize when signals are incomplete or when domain constraints are implicit, making it hard to guarantee that explanations are evidence-backed. KORAL addresses this gap by grounding generation in two explicit KGs and performing structured retrieval over them, enabling citations-to-evidence style answers that unify telemetry, environment, workload context, and prior studies across descriptive, predictive, prescriptive, and what-if SSD queries.

VI. Limitations

KORAL provides exploratory analysis over scattered telemetry and shows promising results, yet important limitations remain. Our evaluation draws on controlled environmental studies and production telemetry that do not cover every deployment setting, and ground truth for rare failures and performance deltas is sparse and imbalanced. Operator query logs are limited, so some what-if and descriptive tasks are under represented. The prescriptions have not been tested in production, so some recommendations may be incorrect. We did not evaluate open source LLMs and relied on a proprietary model, which can introduce training data bias and leaves the performance of KORAL with open source models unknown.

VII. Conclusion

This study presented KORAL, a knowledge driven framework that combines LLMs with two KGs to support SSD operational analysis from fragmented evidence. We address RQ1 by extracting and organizing evidence backed facts from prior studies into a Literature KG, so established results remain reusable during operation. We address RQ2 by converting production telemetry into a structured Data KG that preserves temporal and fleet context for summarization and prediction. We address RQ3 by grounding generation in graph retrieval and domain constraints, which improves faithfulness and reduces unsupported claims. We address RQ4 by enabling grounded descriptive, predictive, prescriptive, and what-if analysis in a single workflow, with evaluations showing that KORAL produces actionable insights with transparent supporting evidence. Although we focus on SSDs, the architecture is not device specific, and it can be adapted to other devices such as GPUs by swapping the ontology and telemetry sources.

Next, we will broaden coverage by incorporating in rack sensors, additional vendors and device models, and continuous production telemetry. We will expand expert review, adopt active labeling, and track long horizon cohorts to improve coverage and reduce bias. We will compare multiple open source language models with the current model and apply domain specific fine tuning, instruction tuning, and calibration to improve faithfulness and robustness. Finally, we will enlarge the query set with operator authored tasks, collect real operator queries, and evaluate prescriptive and what-if tasks more systematically.

Acknowledgments

This work was supported by the following NSF grants: CSR-2402328, CAREER-2338457, CSR-2406069, CSR-2323100, HRD-2225201, 2529283 and 2242700, as well as by the DOE Office of Science User Facility, supported by the Office of Science of the U.S. Department of Energy under Contract No. DE-AC02-05CH11231, using NERSC award DDR-ERCAP0035598. This research was also supported by the U.S. Department of Energy, Office of Science, Advanced Scientific Computing Research, through the SciDAC-RAPIDS2 institute under Contract DE-AC02-06CH11357 and Laboratory Directed Research and Development (LDRD) funding from Argonne National Laboratory under Contract DE-AC02-06CH11357. We thank Nityam Sharadkumar Bhimani (Vertex Pharmaceuticals) for inspiring the core idea of leveraging knowledge graphs to guide and constrain LLM reasoning.

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] S. Liang, Z. Qiao, J. Hochstetler, S. Huang, S. Fu, W. Shi, D. Tiwari, H.-B. Chen, B. Settlemyer, and D. Montoya, "Reliability characterization of solid state drives in a scalable production datacenter," in *2018 IEEE International Conference on Big Data (Big Data)*, 2018, [Online]. Available: https://doi.org/10.1109/BigData.2018.8622643.

[2] A. S. Bankar, S. Sha, J. Bhimani, V. Chaturvedi, and G. Quan, "Thermal aware system-wide reliability optimization for automotive distributed computing applications," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 10, 2022, [Online]. Available: https://doi.org/10.1109/TVT.2022.3185978.

[3] E. P. Wilcox, A. Wood, G. Allen, M. Carts, and M. Casey, "Solid state drive radiation assurance with active testing," in *IEEE Nuclear and Space Radiation Effects Conference (NSREC)*, 2024, [Online]. Available: https://ntrs.nasa.gov/citations/20240008528.

[4] S. Han, P. P. C. Lee, F. Xu, Y. Liu, C. He, and J. Liu, "An in-depth study of correlated failures in production ssd-based data centers," in *19th USENIX Conference on File and Storage Technologies (FAST '21)*, 2021, [Online]. Available: https://www.usenix.org/conference/fast21/presentation/han.

[5] J. Bhimani, T. Patel, N. Mi, and D. Tiwari, "What does vibration do to your ssd?" in *Proceedings of the 56th Annual Design Automation Conference 2019 (DAC '19)*, 2019, [Online]. Available: https://doi.org/10.1145/3316781.331793.

[6] A. Maruf, S. Brahmakshatriya, B. Li, D. Tiwari, G. Quan, and J. Bhimani, "Do temperature and humidity exposures hurt or benefit your ssds?" in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2022, [Online]. Available: https://doi.org/10.23919/DATE54114.2022.9774582.

[7] J. Sarkar and F.-B. F. Sun, "Reliability characterization and modeling of solid-state drives," in *2015 Annual Reliability and Maintainability Symposium (RAMS)*, 2015, [Online]. Available: https://doi.org/10.1109/RAMS.2015.7105166.

[8] J. Meza, Q. Wu, S. Kumar, and O. Mutlu, "A large-scale study of flash memory failures in the field," *ACM SIGMETRICS Performance Evaluation Review*, vol. 43, no. 1, pp. 177–190, 2015, [Online]. Available: https://doi.org/10.1145/2796314.2745848.

[9] B. Schroeder, R. Lagisetty, and A. Merchant, "Flash reliability in production: The expected and the unexpected," in *Proceedings of the 14th Usenix Conference on File and Storage Technologies (FAST '16)*, 2016, pp. 67–80, [Online]. Available: https://www.usenix.org/conference/fast16/technical-sessions/presentation/schroeder.

[10] S.-C. Lin, Y. Zhang, C.-H. Hsu, M. Skach, M. E. Haque, L. Tang, and J. Mars, "The architectural implications of autonomous driving: Constraints and acceleration," in *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '18)*, 2018, pp. 751–766, [Online]. Available: https://doi.org/10.1145/3173162.3173191.

[11] E. Xu, M. Zheng, F. Qin, Y. Xu, and J. Wu, "Lessons and actions: What we learned from 10k ssd-related storage system failures," in *Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference (USENIX ATC '19)*, 2019, pp. 961–975, [Online]. Available: https://www.usenix.org/conference/atc19/presentation/xu.

[12] B. S. Kim, J. Choi, and S. L. Min, "Design tradeoffs for ssd reliability," in *Proceedings of the 17th USENIX Conference on File and Storage Technologies (FAST '19)*, 2019, [Online]. Available: https://www.usenix.org/conference/fast19/presentation/kim-bryan.

[13] G. Yadgar, M. Gabel, S. Jaffer, and B. Schroeder, "Ssd-based workload characteristics and their performance implications," *ACM Transactions on Storage (TOS)*, vol. 17, no. 1, pp. 1–26, 2021, [Online]. Available: https://doi.org/10.1145/3423137.

[14] R. Frickey and J. Doller, "Comparing the reliability of solid-state drives based on tlc and qlc nand flash memories," in *Proceedings of the 2024 IEEE International Reliability Physics Symposium (IRPS)*, 2024, [Online]. Available: https://doi.org/10.1109/IRPS48228.2024.10529487.

[15] Y. Sun, B. Liu, M. Li, Z. Cai, Q. Shi, and Z. Wang, "Modeling retention errors of 3d nand flash for performance and reliability estimation," *ACM Transactions on Storage*, 2024, [Online]. Available: https://dl.acm.org/doi/10.1145/3659101.

[16] J. He, S. Kannan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "The unwritten contract of solid state drives," in *Proceedings of the Twelfth European Conference on Computer Systems (EuroSys '17)*, 2017, pp. 127–141, [Online]. Available: https://doi.org/10.1145/3064176.306418.

[17] K. Wu, A. Arpaci-Dusseau, and R. Arpaci-Dusseau, "Towards an unwritten contract of intel optane ssd," in *Proceedings of the 11th USENIX Conference on Hot Topics in Storage and File Systems (HotStorage '19)*, 2019, p. 3, [Online]. Available: https://www.usenix.org/system/files/hotstorage19-paper-wu-kan.pdf.

[18] D. Li, Q. Wang, and J. Huang, "Learning to drive software-defined solid-state drives," in *Proceedings of the 56th IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*, 2023, [Online]. Available: https://jianh.web.engr.illinois.edu/papers/autoblox-micro23.pdf.

[19] G. Haas *et al.*, "Ssd-iq: Uncovering the hidden side of ssd performance," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 18, no. 12, pp. 4295–4308, 2025, [Online]. Available: https://www.vldb.org/pvldb/vol18/p4295-haas.pdf.

[20] R. Lu, E. Xu, Y. Zhang, Z. Zhu, M. Wang, G. Xue, M. Li, and J. Wu, "Nvme ssd failures in the field: The fail-stop and the fail-slow," in *2022 USENIX Annual Technical Conference (USENIX ATC '22)*, 2022, [Online]. Available: https://www.usenix.org/conference/atc22/presentation/lu.

[21] IBM, "Ssd vs. nvme: What's the difference?" 2023, [Online]. Available: https://www.ibm.com/think/topics/ssd-vs-nvme.

[22] E. S. Forum, "Nvme vs sata: Comparing storage technologies," 2023, [Online]. Available: https://www.enterprisestorageforum.com/hardware/nvme-vs-sata-comparing-storage-technologies/.

[23] Y. Zhou, J. Sun, Y. Li, and X. Zhang, "Learning to coordinate read-write cache policies in ssd," in *Proceedings of the 38th International Conference on Massive Storage Systems and Technology (MSST)*, 2024, [Online]. Available: https://www.msstconference.org/MSST-history/2024/Papers/msst24-6.1.pdf.

[24] I. Memory, "What is power loss protection (plp)?" 2023, [Online]. Available: https://www.intelligentmemory.com/company/articles-news-events/what-is-power-loss-protection-plp.

[25] T. Ren *et al.*, "Device-level optimization techniques for solid-state drives," *arXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2507.10573.

[26] A. Alahmadi *et al.*, "Crash recovery techniques for flash storage devices: A comprehensive survey," *Electronics*, vol. 12, no. 6, p. 1422, 2023, [Online]. Available: https://www.mdpi.com/2079-9292/12/6/1422.

[27] A. Klein, "The ssd edition: 2023 drive stats mid-year review," https://www.backblaze.com/blog/ssd-edition-2023-mid-year-drive-stats-review/, 2023, q1 AFR 0.96% and Q2 AFR 1.05% for SSDs.

[28] ——, "Backblaze drive stats for q1 2023," https://www.backblaze.com/blog/backblaze-drive-stats-for-q1-2023/, 2023, quarterly AFR 1.54%; manufacturer notes incl. Seagate cohorts.

[29] ——, "Backblaze drive stats for q2 2023," https://www.backblaze.com/blog/backblaze-drive-stats-for-q2-2023/, 2023, quarterly AFR 2.28% and discussion of quarterly drift.

[30] Q. Li, H. Li, and K. Zhang, "A survey of ssd lifecycle prediction," in *2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS)*, 2019, [Online]. Available: https://doi.org/10.1109/ICSESS47205.2019.9040759.

[31] Y. He, S. Liang, and S. Fu, "Impact of environmental factors on flash storage performance in autonomous vehicles: An empirical and analytical study," in *2024 IEEE International Conference on Mobility, Operations, Services and Technologies (MOST)*, 2024, [Online]. Available: https://doi.org/10.1109/MOST60774.2024.00029.

[32] S. Ahmadian, F. Taheri, M. Lotfi, M. Karimi, and H. Asadi, "Investigating power outage effects on reliability of solid-state drives," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2018, [Online]. Available: https://doi.org/10.23919/DATE.2018.8342004.

[33] A. Zuck, P. Gühring, T. Zhang, D. E. Porter, and D. Tsafrir, "Why and how to increase ssd performance transparency," in *HotOS '19: Proceedings of the Workshop on Hot Topics in Operating Systems*, 2019, pp. 192–200, [Online]. Available: https://doi.org/10.1145/3317550.3321430.

[34] D. Roy, X. Zhang, R. Bhave, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan, "Exploring llm-based agents for root cause analysis," in *FSE 2024: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, [Online]. Available: https://doi.org/10.1145/3663529.3663841.

[35] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen, "Rcagent: Cloud root cause analysis by autonomous agents

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

with tool-augmented large language models," in *arXiv*, 2023, [Online]. Available: https://doi.org/10.1145/3627673.3680016.

[36] W. Guan, J. Cao, S. Qian, J. Gao, and C. Ouyang, "Logllm: Log-based anomaly detection using large language models," in *arXiv*, 2024, [Online]. Available: https://arxiv.org/abs/2411.08561.

[37] J. Park, B. Choi, C. Lee, and D. Han, "Graph neural network-based slo-aware proactive resource autoscaling framework for microservices," *IEEE/ACM Transactions on Networking*, vol. 32, no. 4, pp. 3331–3346, 2024, [Online]. Available: https://ina.kaist.ac.kr/assets/bibliography/GRAF_ton.pdf.

[38] M. Savasci, A. Souza, L. Wu, D. Irwin, A. Ali-Eldin, and P. Shenoy, "Slo-power: Slo and power-aware elastic scaling for web services," in *Proc. IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2024, [Online]. Available: https://lass.cs.umass.edu/papers/pdf/ccgrid24-slopower.pdf.

[39] J. Zhao, R. Shu, K. Lim, Z. Fan, T. Anderson, M. Gao, and N. E. Jerger, "Arcus: Slo management for accelerators in the cloud with traffic shaping," *arXiv*, 2024, [Online]. Available: https://arxiv.org/abs/2410.17577.

[40] "Self-monitoring, analysis and reporting technology (smart)," 2023, [Online]. Available: https://en.wikipedia.org/wiki/Self-Monitoring,_Analysis_and_Reporting_Technology.

[41] S. Maneas, K. Mahdaviani, T. Emami, and B. Schroeder, "Operational characteristics of ssds in enterprise storage systems: A large-scale field study," in *Proceedings of the 20th USENIX Conference on File and Storage Technologies (FAST '22)*, 2022, [Online]. Available: https://www.usenix.org/conference/fast22/presentation/maneas.

[42] J. Li, B. Huang, Z. Sha, Z. Cai, J. Liao, B. Gerofi, and Y. Ishikawa, "Mitigating negative impacts of read disturb in ssds," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 26, no. 1, pp. 1–24, 2020, [Online]. Available: https://doi.org/10.1145/3410313.

[43] W. Hao, B. Niu, Y. Luo, K. Liu, and N. Liu, "Improving accuracy and adaptability of ssd failure prediction in hyper-scale data centers," *SIGMETRICS Performance Evaluation Review*, vol. 49, no. 4, pp. 99–104, 2022, [Online]. Available: https://doi.org/10.1145/3543146.3543169.

[44] J. Alter, J. Xue, A. Dimnaku, and E. Smirni, "Ssd failures in the field: Symptoms, causes, and prediction models," in *SC '19: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2019, pp. 1–14, [Online]. Available: https://doi.org/10.1145/3295500.3356172.

[45] F. Xu, S. Han, P. P. C. Lee, Y. Liu, C. He, and J. Liu, "General feature selection for failure prediction in large-scale ssd deployment," in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2021, [Online]. Available: https://doi.org/10.1109/DSN48987.2021.00039.

[46] Y. Zhang, W. Hao, B. Niu, K. Liu, S. Wang, N. Liu, X. He, Y. Gwon, and C. Koh, "Multi-view feature-based ssd failure prediction: What, when, and why," in *Proceedings of the 21st USENIX Conference on File and Storage Technologies (FAST '23)*, 2023, [Online]. Available: https://www.usenix.org/conference/atc24/presentation/zhang-yuqi.

[47] Y. Zhang, T. Zhang, W. Hao, S. Wang, N. Liu, X. He, Y. Zhang, W. Wang, Y. Cheng, H. Wang, J. Xu, F. Wang, B. Jiang, Y. Gwon, J. Na, Z. Kim, and G. Oh, "Msfrd: Mutation similarity based ssd failure rating and diagnosis for complex and volatile production environments," in *Proceedings of the 2024 USENIX Annual Technical Conference*, 2024, [Online]. Available: https://www.usenix.org/system/files/fast23-zhang-yuqi.pdf.

[48] C. Chakraborttii and H. Litz, "Improving the accuracy, adaptability, and interpretability of ssd failure prediction models," in *SoCC '20: Proceedings of the 11th ACM Symposium on Cloud Computing*, 2020, pp. 120–133, [Online]. Available: https://doi.org/10.1145/3419111.3421300.

[49] S. Maneas, K. Mahdaviani, T. Emami, and B. Schroeder, "A study of ssd reliability in large scale enterprise storage deployments," in *Proceedings of the 18th USENIX Conference on File and Storage Technologies (FAST '20)*, 2020, [Online]. Available: https://www.usenix.org/conference/fast20/presentation/maneas.

[50] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1285–1298, [Online]. Available: https://users.cs.utah.edu/~lifeifei/papers/deeplog.pdf.

[51] B. Xia, Y. Bai, J. Yin, Y. Li, and J. Xu, "Loggan: A log-level generative adversarial network for anomaly detection using permutation event modeling," *Information Systems Frontiers*, 2021, [Online]. Available: https://www.bingoxia.cn/publication/2021-ISF-xia-LogGAN.pdf.

[52] X. Zhang, X. Chai, M. Yu, and D. Qiu, "Anomaly detection model for log based on lstm network and variational autoencoder," in *2023 4th International Conference on Information Science, Parallel and Distributed Systems (ISPDS)*, 2023, [Online]. Available: https://doi.org/10.1109/ISPDS58840.2023.10235370.

[53] H. Jin, G. Papadimitriou†, K. Raghavan, P. Zuk, P. Balaprakash, C. Wang, A. Mandal, and E. Deelman, "Large language models for anomaly detection in computational workflows: From supervised fine-tuning to in-context learning," in *SC '24: Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, 2024, [Online]. Available: https://doi.org/10.1109/SC41406.2024.00098.

[54] Z. Zhang, F. Yang, X. Qin, J. Zhang, Q. Lin, G. Cheng, D. Zhang, S. Rajmohan, and Q. Zhang, "The vision of autonomic computing: Can llms make it a reality?" in *arXiv*, 2024, [Online]. Available: https://doi.org/10.48550/arXiv.2407.14402.

[55] T. Yang, P. Feng, Q. Guo, J. Zhang, X. Zhang, and J. Ning, "Autohma-llm: Efficient task coordination and execution in heterogeneous multi-agent systems using hybrid large language models," in *IEEE Transactions on Cognitive Communications and Networking*, 2025, [Online]. Available: https://doi.org/10.1109/TCCN.2025.3528892.

[56] S. Zhao, Y. Yang, Z. Wang, Z. He, L. K. Qiu, and L. Qiu, "Retrieval augmented generation (rag) and beyond: A comprehensive survey on how to make your llms use external data more wisely," in *arXiv*, 2024, [Online]. Available: https://arxiv.org/html/2409.14924v1.

[57] D. Edge, H. Truong, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," 2024, [Online]. Available: https://arxiv.org/abs/2404.16130.

[58] M. Akewar, G. Quan, S. Madireddy, and J. Bhimani, "Can llms model the environmental impact on ssd?" in *Proceedings of the 17th ACM Workshop on Hot Topics in Storage and File Systems*, ser. HotStorage '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 100–106. [Online]. Available: https://doi.org/10.1145/3736548.3737835

[59] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *NeurIPS 2023*, 2023, [Online]. Available: https://arxiv.org/abs/2302.04761.

[60] G. Hao, J. Wu, Q. Pan, and R. Morello, "Quantifying the uncertainty of llm hallucination spreading in complex adaptive social networks," in *Nature, Scientific Reports*, 2024, [Online]. Available: https://www.nature.com/articles/s41598-024-66708-4.

[61] C. Egersdoerfer, A. Sareen, J. L. Bez, S. Byna, D. Xu, and D. Dai, "Ioagent: Democratizing trustworthy HPC I/O performance diagnosis capability via llms," in *IEEE International Parallel and Distributed Processing Symposium, IPDPS 2025, Milano, Italy, June 3-7, 2025*. IEEE, 2025, pp. 322–334. [Online]. Available: https://doi.org/10.1109/IPDPS64566.2025.00036

[62] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the ACL (TACL)*, 2024, [Online]. Available: https://aclanthology.org/2024.tacl-1.9/.

[63] A. Oarga, A. Oarga, M. Hart, A. M. Bran, M. Lederbauer, and P. Schwaller, "Scientific knowledge graph and ontology generation using open large language models," 2024, [Online]. Available: https://openreview.net/forum?id=kHsfqBhZjC.

[64] "Alibaba dataset: Ssd smart logs," 2020, [Online]. Available: https://github.com/alibaba-edu/dcbrain/blob/master/ssd_smart_logs/readme.md.

[65] Y. Liu, S. Tao, W. Meng, F. Yao, X. Zhao, and H. Yang, "Logprompt: Prompt engineering towards zero-shot and interpretable log analysis," in *ICSE-Companion '24: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 364–365, [Online]. Available: https://doi.org/10.1145/3639478.3643108.

[66] X. Tang, F. Liu, D. Xu, J. Jiang, Q. Tang, and B. Wang, "Llm-assisted reinforcement learning: Leveraging lightweight large language model capabilities for efficient task scheduling in multi-cloud environment," in *IEEE Transactions on Consumer Electronics*, 2025, [Online]. Available: https://doi.org/10.1109/TCE.2024.3524612.

[67] V. Thakkar, M. Sukumar, J. Dai, K. Singh, and Z. Cai, "Can modern llms tune and configure lsm-based key-value stores?" in *HotStorage '24: Proceedings of the 16th ACM Workshop on Hot Topics in Storage and*

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

File Systems, 2024, pp. 116–123, [Online]. Available: https://doi.org/10.1145/3655038.3665954.

[68] C. Egersdoerfer, A. Sareen, J. L. Bez, S. Byna, and D. Dai, "Ion: Navigating the hpc i/o optimization journey using large language models," in *HotStorage '24: Proceedings of the 16th ACM Workshop on Hot Topics in Storage and File Systems*, 2024, pp. 86–92, [Online]. Available: https://doi.org/10.1145/3655038.3665950.

[69] Q. Wei, Y. Li, Z. Chen, Z. Shen, D. Yu, and B. Li, "Managing hybrid solid-state drives using large language models," in *arXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2503.13105.

[70] Y. Wu, S. Yu, and Y. Li, "Log parsing using llms with self-generated in-context learning and self-correction," *arXiv preprint arXiv:2406.03376*, 2024, [Online]. Available: https://arxiv.org/abs/2406.03376.

[71] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *arXiv*, 2019, [Online]. Available: https://arxiv.org/abs/1908.10084.

[72] T. Wang, G. Qi, and T. Wu, "Kgroot: A knowledge graph-enhanced method for root cause analysis in microservices," *Expert Systems with Applications*, 2024, [Online]. Available: https://arxiv.org/abs/2402.13264.

[73] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2025, [Online]. Available: https://arxiv.org/abs/2404.16130.

[74] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proceedings of the 36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022, [Online]. Available: https://arxiv.org/pdf/2201.11903.

[75] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *NeurIPS 2023*, 2023, [Online]. Available: https://arxiv.org/abs/2305.10601.

[76] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, "Graph of thoughts: Solving elaborate problems with large language models," 2023, [Online]. Available: https://arxiv.org/abs/2308.09687.

[77] R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, and M. Zeng, "Automatic prompt optimization with "gradient descent" and beam search," 2023, [Online]. Available: https://arxiv.org/abs/2305.03495.

[78] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, 2024, [Online]. Available: https://aclanthology.org/2024.tacl-1.9/.

[79] L. Huang and et al., "A survey on hallucination in large language models," 2023, [Online]. Available: https://arxiv.org/abs/2311.05232.

[80] D. Brickley and R. V. Guha, "Rdf schema 1.1," W3C Recommendation, February 2014, [Online]. Available: https://www.w3.org/TR/rdf-schema/.

[81] OWL Working Group, "Owl 2 web ontology language document overview," W3C Recommendation, December 2012, [Online]. Available: https://www.w3.org/TR/owl2-overview/.

[82] *Fio - Flexible I/O Tester Rev.3.36 Documentation*, 2023, [Online]. Available: https://fio.readthedocs.io/en/latest/fio_doc.html.

[83] OpenAI, "Gpt-4 technical report," Tech. Rep., 2023, [Online]. Available: https://cdn.openai.com/papers/gpt-4.pdf.

[84] D. M. W. Powers, "Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation," *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.

[85] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.

[86] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002, pp. 311–318.

[87] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Text Summarization Branches Out: Proceedings of the ACL 2004 Workshop*, 2004.

[88] H. Zhang, E. Thompson, and D. Nissim, "Ssd thermal throttling prediction using improved fast prediction model," in *2019 18th IEEE Intersociety Conference on Thermal and Thermomechanical Phenomena in Electronic Systems (ITherm)*, 2019, [Online]. Available: https://doi.org/10.1109/ITHERM.2019.8757319.

[89] Y. Luo, S. Ghose, Y. Cai, E. F. Haratsch, and O. Mutlu, "Heatwatch: Improving 3d nand flash memory device reliability by exploiting self-recovery and temperature awareness," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2018, [Online]. Available: https://doi.org/10.1109/HPCA.2018.00050.

[90] S. Tannu and P. J. Nair, "The dirty secret of ssds: Embodied carbon," *ACM SIGENERGY Energy Informatics Review*, vol. 3, no. 3, pp. 4–9, 2023, [Online]. Available: https://doi.org/10.1145/3630614.3630616.

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 **Stage I (LitKG):** Extract evidence-backed SSD knowledge from papers into a taxonomy-aligned Literature KG (RDF/Turtle) with provenance/confidence.
- C_2 **Stage II (Operational Analysis):** Summarize telemetry into a DataKG, retrieve LitKG evidence, and produce predictive/descriptive/prescriptive/what-if SSD analysis with automatic metrics.
- C_3 **Fleet mode (Table II):** Cohort-level Stage II (e.g., N=100 drives) to compute fleet predictive + grounding metrics for Alibaba SMART, Google SMART, and SMART+Workload.

B. Computational Artifacts

- A_1 **KORAL code (Stage I + Stage II + Fleet mode)** *GitHub Repository: Damrl-lab/KORAL*
- A_2 **Processed experiment inputs:** prepared CSVs for Alibaba SMART [4], Google SMART [44], and SMART+Workload [4], environmental measurements [5], [6] and FIO [82]. (dataset details and download instructions are provided in the cited paper. After downloading the data and placing it in the folders described in the repository, run the data preparation scripts. Dataset samples are available in the repository).

TABLE III: Artifacts and supported paper elements.

ID	Contrib.	Supported paper elements
A_1	C_1, C_2, C_3	Stage I LitKG build; Stage II per-sample pipeline (Table I); fleet pipeline + aggregation (Table II); prompt/response logging; metric computation
A_2	C_2, C_3	Optional: preprocessed CSVs used to reproduce Table I/II without re-running raw preprocessing

II. Artifact Identification

C. Computational Artifact A_1

Relation To Contributions

Artifact A_1 (KORAL code) implements C_1–C_3: LitKG construction from papers, KG-guided operational analysis on telemetry, and fleet/cohort evaluation for Table II.

Expected Results

Expected results. Stage I outputs `stage_I/out/global_knowledge_graph.ttl`. Stage II outputs a run folder under `stage_II/runs/<RUN>/` with responses and metrics; fleet mode additionally outputs `metrics_summary_fleet.json` and (via the provided script) `table_II_fleet_results.csv`.

Expected Reproduction Time (in Minutes)

Expected reproduction time (minutes). Setup 10–20; a small demo run (few papers + ˜100 samples) is typically 30–120, dominated by LLM calls.

Artifact Setup (incl. Inputs)

Hardware/Software (summary):

- Any x86_64 workstation/server; recommended $\geq$16 GB RAM and $\geq$20 GB disk.
- Python 3.10+; venv/conda environment.
- Key deps: `openai`, `rdflib`, `pandas`, `numpy`, `PyPDF2`, `python-dotenv`.
- Requires `OPENAI_API_KEY` (Stage I + Stage II).

Inputs:

- Stage I: folder of papers (`.pdf/.txt/.md`).
- Stage II: prepared CSVs for `SMART_ALIBABA`, `SMART_GOOGLE`, `SMART_WORKLOAD`.
- Rule base: `rule_base.json` (repo root); taxonomy: `taxonomy.json`.

Artifact Execution

Execution workflow (refer to repo README for full commands/paths).

- T_1 **Build LitKG (Stage I)** $\rightarrow$ produces `global_knowledge_graph.ttl`.
- T_2 **Run Stage II per-sample (Table I)** on one dataset CSV.
- T_3 **Run Stage II fleet mode (Table II)** with `--cohort_size` (e.g., 100) and `--num_cohorts`.
- T_4 **Aggregate Table II** across the three datasets (Alibaba/Google/Workload).

```
python -m stage_II.fleet_cli
  --dataset_type SMART_ALIBABA
  --input_csv <CSV>
  --tasks predictive,descriptive,
  prescriptive,whatif
  --cohort_size 100 --num_cohorts 5
  --out_name <RUN>
```

Artifact Analysis (incl. Outputs)

Outputs (what to check).

- **Stage I:** `stage_I/out/global_knowledge_graph.ttl` exists and contains evidence-linked assertions.
- **Stage II:** `stage_II/runs/<RUN>/responses*.jsonl` and metrics summaries (`metrics_summary*.json` / `*.csv`).
- **Table II:** `table_II_fleet_results.csv` created by the provided aggregation script.

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.

D. Computational Artifact A_2

Relation To Contributions

Artifact A_2 (optional processed CSVs) provides the prepared inputs used for the paper's Stage II evaluation (Table I/II). If A_2 is not released, the same CSVs can be generated using the repo's `data_preparation/` scripts (details in the README).

Expected Results

Expected results. Using these CSVs with A_1 reproduces the same class of outputs (per-sample and fleet metrics + Table II aggregation), subject to minor LLM nondeterminism.

Expected Reproduction Time (in Minutes)

Expected reproduction time. Download/unpack 0–5; execution/analysis as in A_1.

Artifact Evaluation (AE)

A. Computational Artifact A_1

Artifact Setup (incl. Inputs)

Installation and compilation.
1) Create venv and install dependencies (see Artifact Setup in AD). 2) Export `OPENAI_API_KEY`. 3) (Stage I) Ensure prompt path is set:

Artifact Execution

Experiment workflow.

- **Step 1 (Stage I):** Place the papers in the input folder and run Stage I to generate the Literature KG outputs.
- **Step 2 (Stage II inputs):** Use the prepared CSVs for Alibaba SMART, Google SMART, and SMART+Workload.
- **Step 3 (Stage II per-sample):** Run Stage II on a dataset CSV and the selected tasks to generate per-sample outputs.
- **Step 4 (Stage II fleet):** Run `stage_II.fleet_cli` with the desired fleet size and `--num_cohorts`.
- **Step 5 (Table II):** Run `stage_II/scripts/run_table2_fleet.py` to generate `table_II_fleet_results.csv`.

Artifact Analysis (incl. Outputs)

Expected results and evaluation methodology.

- **Stage I correctness:** verify that `stage_I/out/global_knowledge_graph.ttl` exists and contains assertions with evidence/provenance.
- **Stage II per-sample:** verify that each run folder includes `responses.jsonl` and `metrics_summary.json`.
- **Stage II fleet:** verify that `metrics_summary_fleet.json` reports fleet precision/recall/accuracy and grounding metrics (FiP/CFV) for the cohort. For predictive, the pipeline compares the cohort GT labels with the LLM-returned `predicted_failing_drives` list.
- **Table II reproduction:** verify that `table_II_fleet_results.csv` is created and contains one row per dataset type summarizing fleet metrics.
- **Prompt/response integrity:** confirm prompts and raw responses are saved (JSONL). Count JSON parse failures and report them as a run-level statistic.
- **LitKG retrieval sanity:** spot-check that retrieved evidence is non-empty and relevant (e.g., workload evidence appears for SMART+Workload runs).
- **Grounding checks (FiP/CFV):** verify that cited `ref_ids` exist in the available reference set (DataKG refs + retrieved LitKG IDs). For what-if, check that counterfactual statements include directionality and evidence.
- **Fleet predictive mapping:** verify that `predicted_failing_drives` is treated as the positive set and all other cohort drives as negative. Validate `cohort_composition.csv` matches the cohort used for inference.
- **Metric recomputation:** recompute P/R/A from TP/FP/FN/TN and confirm it matches the reported summary. If regression targets exist (e.g., TTF), verify MSE is computed only where both GT and predictions are present.

These outputs substantiate C_1 (LitKG extraction), C_2 (KG-guided operational analysis), and C_3 (fleet-level evaluation).

B. Computational Artifact A_2

Artifact Setup (incl. Inputs)

Setup. Download/unpack processed CSVs into `dataset/` paths described in the repo README. Ensure `taxonomy.json`, `global_knowledge_graph.ttl`, and `rule_base.json` are accessible from repo root (or adjust Stage II config).

Artifact Execution

Execution. Run the same per-sample and fleet commands as in AE for A_1, but skip raw preprocessing. Use the provided CSVs as `--input_csv` inputs.

Artifact Analysis (incl. Outputs)

Analysis. Compare the generated Table II fleet metrics and the per-sample metrics summaries to the numbers reported in the paper.

- **Path verification:** confirm processed CSVs are placed under the expected `dataset/` paths and Stage II can locate `taxonomy.json`, `global_knowledge_graph.ttl`, and `rule_base.json` (or use config overrides).
- **Schema validation:** verify SMART columns (`r_<id>`) and a GT label column (`failure` or `label`) exist. For SMART+Workload, confirm `app` is present and populated.
- **Reproduction fidelity:** run the same cohort size, number of cohorts, seed, tasks, and temperature as the paper's Table II setup; compare the generated CSV to the reported values (allowing small LLM variance).

Authorized licensed use limited to: FLORIDA INTERNATIONAL UNIVERSITY. Downloaded on October 22,2026 at 19:51:42 UTC from IEEE Xplore. Restrictions apply.